OBSERVACIONES DE LA PRÁCTICA

Alvaro Villate - 202616607
Nicolas Liz Figueroa - 202611971
Juan Pablo Martinez - 202611361

Preguntas de análisis

1. ¿Qué diferencia existe entre las alturas de los dos árboles (BST y RBT)?

   El árbol de fechas tiene 1177 elementos en los dos laboratorios, pero la altura cambia mucho: el BST del laboratorio pasado quedó con altura 30 y el RBT de este laboratorio quedó con altura 13. Con 1177 llaves, un árbol perfectamente balanceado tendría una altura cercana a $\log_2(1177) \approx 10$, así que el RBT quedó bastante cerca de lo ideal y el BST estaba casi al triple. La diferencia se debe a que el BST depende del orden en que llegan las fechas, que vienen casi en orden cronológico y lo hacen crecer hacia un solo lado. El RBT, en cambio, se reequilibra con rotaciones y cambios de color en cada inserción, y eso garantiza que la altura nunca pase de $2 \cdot \log_2(n+1)$, que para este caso es aproximadamente 20. Por eso el 13 que obtuvimos está dentro de lo esperado.

2. ¿Percibe alguna diferencia entre la ejecución de los dos árboles (RBT y BST)? ¿Por qué pasa esto?

   Con este tamaño de datos la diferencia de tiempo no se nota mucho a simple vista, pero el RBT debería ser igual de rápido o más rápido que el BST. Por cada crimen que se carga se hace una búsqueda en el árbol de fechas (get), y solo se inserta (put) cuando la fecha es nueva. Son 319073 crímenes y solo 1177 fechas distintas, así que casi todo el tiempo de carga se va en búsquedas y no en inserciones. En esas búsquedas el RBT recorre máximo unos 13 niveles, mientras que el BST puede recorrer hasta 30, o sea que hace menos comparaciones por cada crimen. Lo que sí le cuesta más al RBT son las inserciones, porque hace rotaciones y cambios de color para mantenerse balanceado, pero eso pasa solo 1177 veces y casi no pesa frente a lo que ahorra en las búsquedas. Lo mismo aplica para las consultas por rango, que dependen de la altura del árbol.

3. ¿Existe alguna diferencia de complejidad entre los dos árboles (RBT y BST)? Justifique su respuesta.

   Sí. En un BST las operaciones put, get y contains cuestan O(h), donde h es la altura del árbol. Como el BST no se balancea, h puede ir desde $\log_2(n)$ hasta n; en el peor caso (llaves insertadas ya ordenadas) el árbol se vuelve una lista enlazada y las operaciones cuestan O(n). En un RBT la altura está acotada por $2 \cdot \log_2(n+1)$, así que put, get y contains cuestan O(log n) siempre, incluso en el peor caso. Las consultas por rango (keys y values) pasan de O(h + k) a O(log n + k), donde k es el número de llaves dentro del rango. Es cierto que un BST con llaves en orden aleatorio también se comporta en promedio como O(log n), pero el RBT lo asegura sin importar el orden de llegada. A cambio, el RBT hace trabajo extra en cada put (rotaciones y flip de colores), pero son operaciones O(1) que no cambian el orden O(log n).

4. ¿Existe alguna manera de cargar los datos en un árbol RBT de tal forma que su funcionamiento mejore? Si es así, mencione cuál.

   Sí, hay dos formas. La primera es insertar las llaves en orden aleatorio y no cronológico: cuando las llaves llegan ya ordenadas, el RBT tiene que hacer muchas rotaciones para mantener el balance, y con un orden aleatorio se necesitan menos. La segunda, si todos los datos están disponibles desde el principio, es ordenar las llaves primero y construir el árbol directamente balanceado a partir de la lista ordenada (la mitad queda como raíz y se repite lo mismo a cada lado, pintando de rojo los nodos del último nivel si no queda completo). Eso cuesta O(n) una vez ordenadas las llaves y evita las rotaciones por completo.